LegalRAG – Coding and Implementation

(BY: E0123020, E0123024, E0123051)

settings.py:

```
    """
Central configuration file for LegalRAG.
Modify values here instead of changing them
throughout the project.
"""

# ==========================
# Retrieval Settings
# ==========================

TOP_K = 20

DENSE_WEIGHT = 0.65
SPARSE_WEIGHT = 0.35

# ==========================
# Context Builder
# ==========================

MAX_CONTEXT_CHARS = 6000

# ==========================
# LLM Settings
# ==========================

OLLAMA_URL = "http://localhost:11434/api/generate"

OLLAMA_MODEL = "llama3:8b"

TEMPERATURE = 0.1

MAX_GENERATION_TOKENS = 300

# ==========================
# Logging
# ==========================

LOG_LEVEL = "INFO"

# ==========================
```

```
# Future Settings
# ========================

ENABLE_RERANKING = False

ENABLE_QUERY_EXPANSION = False
```

scraper.py:

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import time
import re

from configs.scraper_queries import LEGAL_SEARCH_QUERIES

BASE_URL = "https://indiankanoon.org"

HEADERS = {
   "User-Agent": "Mozilla/5.0"
}

# ------------------------------
# STEP 1: GET SEARCH PAGE LINKS
# ------------------------------
def get_case_links(search_url: str, limit: int = 20):

   print(f"\nFetching search page:\n{search_url}\n")

   response = requests.get(search_url, headers=HEADERS)

   print("Search page status:", response.status_code)

   if response.status_code != 200:
      return []

   soup = BeautifulSoup(response.text, "lxml")

   # DEBUG: confirm page is correct
   print("\nPage title:", soup.title.text if soup.title else "No title")

   links = []
```

```
    # STRATEGY 1: extract all /doc/ links (MOST RELIABLE)
    for a in soup.find_all("a", href=True):
        href = a["href"]

        if "/doc/" in href:
            full_url = BASE_URL + href
            if full_url not in links:
                links.append(full_url)

        if len(links) >= limit:
            break

    print(f"\nExtracted {len(links)} case links")

    return links

# ------------------------------
# STEP 2: SCRAPE SINGLE CASE
# ------------------------------
def scrape_case(case_url: str):

    print(f"\nScraping: {case_url}")

    response = requests.get(case_url, headers=HEADERS)

    print("Status:", response.status_code)

    if response.status_code != 200:
        return None

    soup = BeautifulSoup(response.text, "lxml")

    # ----------------------
    # TITLE EXTRACTION
    # ----------------------
    title = soup.title.text.strip() if soup.title else "Unknown Case"

    bad_keywords = [
        "section",
        "article",
        "constitution",
        "act",
        "rules"
    ]

    if any(word in title.lower() for word in bad_keywords):
        return None
```

```
    # Clean title
    title = re.sub(r"\s+", " ", title)

    # ----------------------
    # TEXT EXTRACTION STRATEGY (ROBUST)
    # ----------------------

    text = ""

    # Strategy 1: known judgment containers
    selectors = [
        "div.judgments",
        "div.judgment",
        "div#judgments",
        "pre"
    ]

    for sel in selectors:
        tag = soup.select_one(sel)
        if tag:
            text = tag.get_text(" ", strip=True)
            break

    # Strategy 2: fallback full page text (LAST RESORT)
    if not text or len(text) < 200:
        text = soup.get_text(" ", strip=True)

    # ----------------------
    # CLEANING
    # ----------------------
    text = re.sub(r"\s+", " ", text)

    if len(text) < 300:
        print("Skipping (too short text)")
        return None

    return {
        "case_title": title,
        "source": case_url,
        "text": text
    }

# ------------------------------
# STEP 3: MAIN PIPELINE
# ------------------------------
def main():

    dataset = []
```

```
    visited_links = set()

    for query in LEGAL_SEARCH_QUERIES:

        print(f"\n========== QUERY: {query} ==========\n")

        search_url = (
            f"https://indiankanoon.org/search/?formInput={query}"
        )

        links = get_case_links(search_url, limit=10)

        print(f"Found {len(links)} links")

        for link in links:

            if link in visited_links:
                continue

            visited_links.add(link)

            try:

                case_data = scrape_case(link)

                if case_data:
                    dataset.append(case_data)

            except Exception as e:
                print("Error:", e)

            time.sleep(1.5)

    print(f"\nFinal dataset size: {len(dataset)}")

    df = pd.DataFrame(dataset)

    df.to_csv("data/raw/judgments.csv", index=False)

    print("\nDataset saved successfully.")

if __name__ == "__main__":
    main()
```

clean_corpus.py:

```
import pandas as pd
import re

MIN_TEXT_LENGTH = 500

def normalize_text(text: str) -> str:
    """
    Clean and normalize legal text.
    """

    if not isinstance(text, str):
        return ""

    # Remove excessive whitespace
    text = re.sub(r"\s+", " ", text)

    # Remove weird unicode characters
    text = text.replace("\xa0", " ")

    # Strip leading/trailing spaces
    text = text.strip()

    return text

def is_valid_case(title: str) -> bool:
    """
    Keep likely judicial cases only.
    """

    if not isinstance(title, str):
        return False

    title = title.lower()

    blacklist = [
        "constitution of india",
        "section ",
        "article ",
        "act,",
```

```
        "rules,"
    ]

    for word in blacklist:
        if word in title:
            return False

    return True

def clean_dataset(input_path: str, output_path: str):

    print("\nLoading dataset...")

    df = pd.read_csv(input_path)

    print(f"Original dataset size: {len(df)}")

    # ---------------------------------
    # REMOVE NULLS
    # ---------------------------------
    df = df.dropna(subset=["text"])

    print(f"After null removal: {len(df)}")

    # ---------------------------------
    # NORMALIZE TEXT
    # ---------------------------------
    print("\nNormalizing text...")

    df["text"] = df["text"].apply(normalize_text)

    # ---------------------------------
    # REMOVE SHORT DOCUMENTS
    # ---------------------------------
    print("\nRemoving short cases...")

    df = df[df["text"].str.len() > MIN_TEXT_LENGTH]

    print(f"After short text removal: {len(df)}")

    # ---------------------------------
    # REMOVE DUPLICATES
    # ---------------------------------
    print("\nRemoving duplicate cases...")

    before = len(df)

    df = df.drop_duplicates(subset=["text"])
```

```
    after = len(df)

    print(f"Removed {before - after} duplicate cases")

    # ---------------------------------
    # RESET INDEX
    # ---------------------------------
    df = df.reset_index(drop=True)

    # ---------------------------------
    # SAVE CLEANED DATASET
    # ---------------------------------
    df.to_csv(output_path, index=False)

    print(f"\nCleaned dataset saved to:\n{output_path}")

    print("\nFinal dataset preview:\n")
    print(df.head())

def main():

    input_path = "data/raw/judgments.csv"

    output_path = "data/cleaned/cleaned_judgments.csv"

    clean_dataset(input_path, output_path)

if __name__ == "__main__":
    main()
```

chunker.py:

```
import pandas as pd
import re
from typing import List

CHUNK_SIZE = 700
OVERLAP = 120
MIN_CHUNK_WORDS = 100

def split_into_words(text: str) -> List[str]:
    return text.split()
```

```
def create_chunks(words: List[str], chunk_size: int, overlap: int):

    chunks = []
    start = 0

    while start < len(words):

        end = start + chunk_size
        chunk_words = words[start:end]

        # Handle very small final chunks
        if len(chunk_words) < MIN_CHUNK_WORDS:

            if chunks:
                chunks[-1] += " " + " ".join(chunk_words)
            else:
                chunks.append(" ".join(chunk_words))

            break

        chunk_text = " ".join(chunk_words)
        chunks.append(chunk_text)

        start += chunk_size - overlap

    return chunks

def chunk_document(text: str):

    words = split_into_words(text)

    chunks = create_chunks(
        words,
        CHUNK_SIZE,
        OVERLAP
    )

    return chunks

def process_dataset(input_path: str, output_path: str):

    print("\nLoading cleaned dataset...")

    df = pd.read_csv(input_path)

    print(f"Loaded {len(df)} documents")

    chunked_rows = []
```

```
    total_chunks = 0

    for idx, row in df.iterrows():

        case_title = row["case_title"]
        source = row["source"]
        text = row["text"]

        chunks = chunk_document(text)

        print(f"\nProcessing document {idx+1}")
        print(f"Generated {len(chunks)} chunks")

        for chunk_id, chunk_text in enumerate(chunks):

            chunked_rows.append({
                "case_title": case_title,
                "source": source,
                "chunk_id": chunk_id,
                "chunk_text": chunk_text
            })

        total_chunks += len(chunks)

    chunk_df = pd.DataFrame(chunked_rows)

    print(f"\nTotal chunks created: {total_chunks}")

    chunk_df.to_csv(output_path, index=False)

    print(f"\nChunked dataset saved to:\n{output_path}")

    print("\nSample chunks:\n")
    print(chunk_df.head())

def main():

    input_path = "data/cleaned/cleaned_judgments.csv"

    output_path = "data/chunks/chunked_judgments.csv"

    process_dataset(input_path, output_path)

if __name__ == "__main__":
    main()
```

sparse_retriever.py:

```
import joblib

from sklearn.metrics.pairwise import (
    cosine_similarity
)

class SparseRetriever:

    def __init__(self):

        db = joblib.load(
            "vector_db/tfidf_store.joblib"
        )

        self.vectorizer = db["vectorizer"]
        self.matrix = db["matrix"]
        self.metadata = db["metadata"]

    def search(self, query, top_k=5):

        query_vector = (
            self.vectorizer.transform([query])
        )

        similarities = cosine_similarity(
            query_vector,
            self.matrix
        )[0]

        top_indices = (
            similarities.argsort()[::-1][:top_k]
        )

        results = []

        for idx in top_indices:

            row = self.metadata.iloc[idx]

            results.append({
                "score": float(
```

```
            similarities[idx]
        ),
        "case_title": row["case_title"],
        "source": row["source"],
        "chunk_text": row["chunk_text"]
      })

    return results
```

dense_retriever.py:

```
import joblib
import torch

from transformers import (
  AutoTokenizer,
  AutoModel
)

MODEL_NAME = (
  "nlpaueb/legal-bert-base-uncased"
)

class DenseRetriever:

  def __init__(self):

    print("Loading vector DB...")

    db = joblib.load(
      "vector_db/nearest_neighbors.joblib"
    )

    self.index = db["index"]
    self.metadata = db["metadata"]

    print("Loading LegalBERT...")

    self.tokenizer = (
      AutoTokenizer.from_pretrained(
        MODEL_NAME
      )
```

```
    )

    self.model = (
        AutoModel.from_pretrained(
            MODEL_NAME
        )
    )

    self.model.eval()

@torch.no_grad()
def embed_query(self, query):

    encoded = self.tokenizer(
        query,
        truncation=True,
        padding=True,
        max_length=512,
        return_tensors="pt"
    )

    outputs = self.model(**encoded)

    embeddings = outputs.last_hidden_state

    mask = (
        encoded["attention_mask"]
        .unsqueeze(-1)
        .expand(embeddings.size())
        .float()
    )

    pooled = (
        embeddings * mask
    ).sum(1) / mask.sum(1)

    return pooled.numpy()

def search(self, query, top_k=5):

    query_vector = (
        self.embed_query(query)
    )

    distances, indices = (
        self.index.kneighbors(
            query_vector,
            n_neighbors=top_k
```

```
        )
    )

    results = []

    for score, idx in zip(
        distances[0],
        indices[0]
    ):

        row = self.metadata.iloc[idx]

        results.append({
            "score": 1 - float(score),
            "case_title": row["case_title"],
            "source": row["source"],
            "chunk_text": row["chunk_text"]
        })

    return results
```

hybrid_retriever.py:

```
from collections import defaultdict

from src.retrieval.dense_retriever import DenseRetriever
from src.retrieval.sparse_retriever import SparseRetriever
from src.retrieval.cross_encoder_reranker import CrossEncoderReranker

DENSE_WEIGHT = 0.65
SPARSE_WEIGHT = 0.35

RERANK_CANDIDATES = 20

class HybridRetriever:

    def __init__(self):

        self.dense = DenseRetriever()
        self.sparse = SparseRetriever()
        self.reranker = CrossEncoderReranker()

    def search(self, query, top_k=5):

        # ---------------------------------
```

```
# Dense Retrieval
# ---------------------------------

dense_results = self.dense.search(
   query,
   top_k=50
)

# ---------------------------------
# Sparse Retrieval
# ---------------------------------

sparse_results = self.sparse.search(
   query,
   top_k=50
)

combined = defaultdict(dict)

# ---------------------------------
# Dense Scores
# ---------------------------------

for result in dense_results:

   key = (
      result["source"],
      result["chunk_text"]
   )

   combined[key]["dense"] = (
      result["score"]
   )

   combined[key]["data"] = result

# ---------------------------------
# Sparse Scores
# ---------------------------------

for result in sparse_results:

   key = (
      result["source"],
      result["chunk_text"]
   )

   combined[key]["sparse"] = (
```

```
        result["score"]
    )

    combined[key]["data"] = result

# ---------------------------------
# Hybrid Fusion
# ---------------------------------

final_results = []

for item in combined.values():

    dense_score = item.get(
        "dense",
        0
    )

    sparse_score = item.get(
        "sparse",
        0
    )

    hybrid_score = (
        DENSE_WEIGHT * dense_score
        +
        SPARSE_WEIGHT * sparse_score
    )

    data = item["data"]

    data["hybrid_score"] = (
        hybrid_score
    )

    final_results.append(data)

# ---------------------------------
# Sort by Hybrid Score
# ---------------------------------

final_results.sort(
    key=lambda x: x["hybrid_score"],
    reverse=True
)

# ---------------------------------
# Remove Duplicate Cases
```

```
# ---------------------------------

unique_results = []
seen_cases = set()

for item in final_results:

    case_title = (
        item["case_title"]
        .strip()
        .lower()
    )

    if case_title in seen_cases:
        continue

    seen_cases.add(case_title)
    unique_results.append(item)

# ---------------------------------
# Select Top Candidates
# ---------------------------------

candidate_pool = unique_results[
    :RERANK_CANDIDATES
]

print(
    f"\nCandidate Pool Size: "
    f"{len(candidate_pool)}"
)

# ---------------------------------
# Neural Reranking
# ---------------------------------

reranked_results = (
    self.reranker.rerank(
        query=query,
        candidates=candidate_pool
    )
)

# ---------------------------------
# Final Top-K
# ---------------------------------

final_topk = reranked_results[:top_k]
```

```
# ---------------------------------
# Statistics
# ---------------------------------

print("\nRetrieval Statistics")

print(
    f"Unique Cases Retrieved: "
    f"{len(final_topk)}"
)

if final_topk:

    scores = [
        item.get("final_score",0)
        for item in final_topk
    ]

    avg_score = (
        sum(scores)
        / len(scores)
    )

    best_score = max(scores)

    print(
        f"Average Score: "
        f"{avg_score:.4f}"
    )

    print(
        f"Best Match Score: "
        f"{best_score:.4f}"
    )

print("\nTOP RETRIEVAL RESULTS")

for i, item in enumerate(
    final_topk,
    start=1
):

    print(
        f"{i}. "
        f"{item['case_title']} "
        f"(Final={item.get('final_score',0):.4f}, "
        f"Cross={item.get('cross_score',0):.4f}, "
```

```
            f"Hybrid={item['hybrid_score']:.4f})"
        )

    return final_topk
```

embedder.py:

```
import pandas as pd
import numpy as np
import torch

from tqdm import tqdm
from transformers import AutoTokenizer, AutoModel
import joblib

MODEL_NAME = "nlpaueb/legal-bert-base-uncased"

DEVICE = (
    "cuda"
    if torch.cuda.is_available()
    else "cpu"
)

class LegalBERTEmbedder:

    def __init__(self):

        print(f"\nLoading model: {MODEL_NAME}")

        self.tokenizer = AutoTokenizer.from_pretrained(
            MODEL_NAME
        )

        self.model = AutoModel.from_pretrained(
            MODEL_NAME
        )

        self.model.to(DEVICE)
        self.model.eval()

        print(f"Using device: {DEVICE}")

    @torch.no_grad()
    def embed_text(self, text: str):
```

```
        encoded = self.tokenizer(
            text,
            truncation=True,
            padding=True,
            max_length=512,
            return_tensors="pt"
        )

        encoded = {
            k: v.to(DEVICE)
            for k, v in encoded.items()
        }

        output = self.model(**encoded)

        token_embeddings = output.last_hidden_state

        attention_mask = (
            encoded["attention_mask"]
            .unsqueeze(-1)
            .expand(token_embeddings.size())
            .float()
        )

        masked_embeddings = (
            token_embeddings * attention_mask
        )

        summed = masked_embeddings.sum(dim=1)

        counts = attention_mask.sum(dim=1)

        mean_pooled = summed / counts

        return mean_pooled.squeeze().cpu().numpy()

def generate_embeddings():

    print("\nLoading chunks...")

    df = pd.read_csv(
        "data/chunks/chunked_judgments.csv"
    )

    print(f"Loaded {len(df)} chunks")

    embedder = LegalBERTEmbedder()
```

```
    embeddings = []

    for chunk in tqdm(df["chunk_text"]):

        vector = embedder.embed_text(chunk)

        embeddings.append(vector)

    embeddings = np.array(embeddings)

    print("\nEmbedding shape:")
    print(embeddings.shape)

    output = {
        "embeddings": embeddings,
        "metadata": df
    }

    joblib.dump(
        output,
        "data/embeddings/legalbert_embeddings.joblib"
    )

    print(
        "\nSaved embeddings to:"
        "\ndata/embeddings/legalbert_embeddings.joblib"
    )

if __name__ == "__main__":
    generate_embeddings()
```

cross_encoder_reranker.py:

```
import numpy as np

from sentence_transformers import CrossEncoder

MODEL_NAME = (
    "cross-encoder/ms-marco-MiniLM-L-6-v2"
)

class CrossEncoderReranker:
```

```
def __init__(self):

    print(
        "\nLoading CrossEncoder..."
    )

    self.model = CrossEncoder(
        MODEL_NAME
    )

def rerank(self, query, candidates):
    pairs = [(
        query,
        item["chunk_text"]
    ) for item in candidates
    ]

    scores = self.model.predict(pairs)


    scores = np.array(scores)

    min_score = scores.min()
    max_score = scores.max()

    scores = (
        (scores - min_score) / (max_score - min_score + 1e-8)
    )

    reranked = []

    for item, score in zip(candidates, scores):
        new_item = item.copy()
        new_item["cross_score"] = (
            float(score)
        )

        new_item["final_score"] = (
            0.7 * new_item["cross_score"]
            +
            0.3 * new_item["hybrid_score"]
        )

        reranked.append(new_item)

    reranked.sort(
        key=lambda x:
        x["final_score"],
```

```
        reverse=True
    )

    return reranked
```

inference.py:

```
from unittest import result
import requests

from src.llm.model_loader import OllamaConfig
from src.utils.logger import logger
from src.config.settings import OLLAMA_URL

class OllamaLLM:

    def __init__(self):

        self.model = (
            OllamaConfig.MODEL_NAME
        )

        self.url = OLLAMA_URL

    def generate(self, prompt: str):

        payload = {
            "model": self.model,
            "prompt": prompt,
            "stream": False,
            "options": {
                "temperature": OllamaConfig.TEMPERATURE,
                "num_predict": OllamaConfig.MAX_TOKENS,
                "stop": [
                    "LEGAL QUESTION",
                    "USER QUESTION",
                    "[CASE",
                    "FINAL LEGAL ANSWER:"
                ]
            }
        }
```

```
logger.info(
    f"Calling Ollama model: {self.model}"
)

try:
    response = requests.post(
        self.url,
        json=payload,
        timeout=300
    )

    if response.status_code != 200:
        print("\nOLLAMA ERROR:")
        print(response.text)

        response.raise_for_status()

    result = response.json()

    print("\n" + "="*80)
    print("RAW OLLAMA JSON")
    print("="*80)
    print(result)

    print("\n" + "="*80)
    print("RAW ANSWER")
    print("="*80)
    print(result.get("response", ""))

    answer = result.get(
        "response",
        ""
    )

    logger.info(
        "Answer generated successfully"
    )

    return answer

except requests.exceptions.RequestException as e:

    logger.error(
        f"Ollama request failed: {e}"
    )

    return (
        f"LLM Error: {e}"
```

)

model_loader.py:

```
import logging

from src.config.settings import OLLAMA_MODEL

logger = logging.getLogger(__name__)

class OllamaConfig:

    MODEL_NAME = OLLAMA_MODEL

    TEMPERATURE = 0.1

    MAX_TOKENS = 1024
```

legal_rag_pipeline.py:

```
from src.retrieval.hybrid_retriever import (
    HybridRetriever
)

from src.reasoning.context_builder import (
    ContextBuilder
)

from src.reasoning.context_compressor import (
    ContextCompressor
)

from src.reasoning.prompt_builder import (
    PromptBuilder
)

from src.llm.inference import (
    OllamaLLM
)

from src.utils.logger import (
    logger
```

```
)

from src.reasoning.citation_generator import (
    CitationGenerator
)

class LegalRAGPipeline:

    def __init__(self):

        self.retriever = (
            HybridRetriever()
        )

        self.compressor = (
            ContextCompressor()
        )

        self.context_builder = (
            ContextBuilder()
        )

        self.llm = (
            OllamaLLM()
        )

    def answer_query(
        self,
        query: str,
        top_k: int = 5
    ):

        logger.info(
            f"Received query: {query}"
        )

        # ------------------
        # Retrieval
        # ------------------

        retrieved_chunks = (
            self.retriever.search(
                query=query,
                top_k=top_k
            )
        )

        if not retrieved_chunks:
```

```
    return {
        "query": query,
        "answer":
            "No relevant legal context found.",
        "sources": []
    }

logger.info(
    f"Retrieved "
    f"{len(retrieved_chunks)} chunks"
)

# ------------------
# Context Compression
# ------------------

compressed_chunks = (
    self.compressor.compress(
        retrieved_chunks
    )
)

# ------------------
# Context Building
# ------------------

print("\nDEBUG RETRIEVED CHUNKS")
print(type(retrieved_chunks))

if retrieved_chunks:
    print(type(retrieved_chunks[0]))
    print(retrieved_chunks[0])

context = (
    self.context_builder
    .build_context(
        retrieved_chunks
    )
)

logger.info(
    f"Context Length: "
    f"{len(context)} chars"
)

# ------------------
# Prompt Building
```

```
# ------------------

prompt = (
    PromptBuilder
    .build_legal_prompt(
        query=query,
        context=context
    )
)

logger.info(
    f"Prompt Length: "
    f"{len(prompt)} chars"
)

logger.info(
    "Sending prompt to LLM"
)

# ------------------
# LLM Generation
# ------------------

answer = (
    self.llm.generate(
        prompt
    )
)

# ------------------
# Citations
# ------------------

citations = (
    CitationGenerator.generate(
        retrieved_chunks
    )
)

# ------------------
# Confidence
# ------------------

top_scores = [
     c["score"]
     for c in citations[:3]
]
```

```
        confidence = round(
            (
                sum(top_scores) / len(top_scores)
            ) * 100,
            2
        )

        logger.info(
            "Answer generated successfully"
        )

        return {

            "query": query,

            "answer": answer,

            "sources": retrieved_chunks,

            "num_sources": len(
                retrieved_chunks
            ),

            "context": context,

            "citations": citations,

            "confidence": confidence
        }
```

context_builder.py:

```
from typing import List, Dict

from src.config.settings import (
    MAX_CONTEXT_CHARS
)

class ContextBuilder:

    def __init__(
        self,
        max_context_chars=MAX_CONTEXT_CHARS
```

```
):

    self.max_context_chars = (
        max_context_chars
    )

# ----------------------------
# Remove duplicates
# ----------------------------

def remove_duplicates(
    self,
    retrieved_chunks: List[Dict]
):

    seen = set()

    unique_chunks = []

    for chunk in retrieved_chunks:

        key = (
            chunk["case_title"],
            chunk["chunk_text"]
        )

        if key not in seen:

            seen.add(key)

            unique_chunks.append(
                chunk
            )

    return unique_chunks

# ----------------------------
# Sort chunks
# ----------------------------

def sort_chunks(
    self,
    retrieved_chunks: List[Dict]
):

    if not retrieved_chunks:
        return []
```

```
        if "final_score" in retrieved_chunks[0]:

            score_field = (
                "final_score"
            )

        elif "cross_score" in retrieved_chunks[0]:

            score_field = (
                "cross_score"
            )

        elif "hybrid_score" in retrieved_chunks[0]:

            score_field = (
                "hybrid_score"
            )

        else:

            score_field = (
                "score"
            )

        return sorted(
            retrieved_chunks,
            key=lambda x:
            x[score_field],
            reverse=True
        )

    # ----------------------------
    # Build Context
    # ----------------------------

    def build_context(
        self,
        retrieved_chunks: List[Dict]
    ):

        retrieved_chunks = (
            self.remove_duplicates(
                retrieved_chunks
            )
        )

        retrieved_chunks = (
            self.sort_chunks(
```

```
            retrieved_chunks
        )
    )

    context_parts = []

    current_length = 0

    for i, chunk in enumerate(
        retrieved_chunks,
        start=1
    ):

        score = chunk.get(
            "final_score",
            chunk.get(
                "cross_score",
                chunk.get(
                    "hybrid_score",
                    chunk.get(
                        "score",
                        0
                    )
                )
            )
        )

        case_block = f"""
[CASE {i}]

Case Title:
{chunk['case_title']}

Score:
{score:.4f}

Source:
{chunk['source']}

Relevant Text:
{chunk['chunk_text']}

"""

        if (
            current_length
            + len(case_block)
            > self.max_context_chars
```

```
            ):
                break

            context_parts.append(
                case_block
            )

            current_length += (
                len(case_block)
            )

        print(
            f"\nContext Length: "
            f"{current_length} chars"
        )

        print(
            f"Cases Used: "
            f"{len(context_parts)}"
        )

        return "\n".join(
            context_parts
        )
```

prompt_builder.py:

```
from src.reasoning.prompt_templates import (
    PromptTemplates
)

class PromptBuilder:

    @staticmethod
    def build_legal_prompt(
        query,
        context
    ):

        prompt = (
            PromptTemplates
            .legal_reasoning_prompt(
                query=query,
                context=context
```

```
        )
    )

    print(
        "\nPrompt Length:",
        len(prompt)
    )

    return prompt
```

prompt_templates.py:

```
class PromptTemplates:

    @staticmethod
    def legal_reasoning_prompt(
        query: str,
        context: str
    ) -> str:

        return f"""
You are an Indian Legal AI Assistant specializing in legal research and case-law reasoning.

You MUST answer ONLY using the legal context provided below.

Do NOT use any external legal knowledge.

If the legal context does not contain enough information to answer the question, respond exactly:

Insufficient legal context available.

==================================================
LEGAL CONTEXT
==================================================

{context}

==================================================
LEGAL QUESTION
==================================================

{query}

==================================================
RULES
```

==========

1. Use ONLY the supplied legal context.
2. Do NOT use external legal knowledge.
3. Do NOT invent facts, statutes, principles, or precedents.
4. Do NOT speculate.
5. Base every conclusion on the retrieved cases.
6. Mention case names when they support a legal principle.
7. Do NOT repeat the question in the analysis.
8. Do NOT repeat the same case multiple times unnecessarily.
9. Synthesize legal principles across cases instead of summarizing each case separately.
10. Keep the answer concise and professional.
11. Do NOT create a memorandum heading, date, subject line, or formal letter structure.

==========
REASONING PROCESS
==========

Step 1:
Identify the legal issue.

Step 2:
Identify the governing legal principles from the retrieved cases.

Step 3:
Identify the most relevant authorities.

Step 4:
Apply those principles to the user's question.

Step 5:
Provide a clear and qualified conclusion.

==========
OUTPUT FORMAT
==========

LEGAL ISSUE:
(1-2 sentences)

RELEVANT AUTHORITIES:
- Case Name
- Case Name

LEGAL REASONING:
(Explain the governing legal principles.
Synthesize the authorities.
Avoid repeating case summaries.

Use only information contained in the context.)

CONCLUSION:
(Directly answer the legal question.
Maximum 4-5 sentences.
Do not repeat the analysis.)

IMPORTANT:
- Do NOT include a SOURCES section.
- Do NOT include a CASE section.
- Do NOT repeat authorities already listed above.
- Avoid redundant statements.
"""

context_compressor.py:

```
class ContextCompressor:

    def __init__(
        self,
        chunk_size=500
    ):
        self.chunk_size = chunk_size

    def compress(
        self,
        retrieved_chunks
    ):

        compressed_parts = []

        for item in retrieved_chunks:

            case_title = item.get(
                "case_title",
                "Unknown Case"
            )

            chunk_text = item.get(
                "chunk_text",
                ""
            )
```

```
            compressed_text = (
                chunk_text[:self.chunk_size]
            )

            compressed_parts.append(
                f"CASE: {case_title}\n"
                f"{compressed_text}\n"
            )

        return "\n".join(
            compressed_parts
        )
```

citation_generator.py:

```
class CitationGenerator:

    @staticmethod
    def generate(
        retrieved_chunks
    ):

        citations = []

        for rank, chunk in enumerate(
            retrieved_chunks,
            start=1
        ):

            citations.append(
                {
                    "rank": rank,

                    "case_title":
                        chunk["case_title"],

                    "source":
                        chunk["source"],

                    "score":
                        round(
                            chunk.get(
                                "hybrid_score",
```

```
                    0
                ),
                4
            )
        }
    )

    return citations
```

app.py:

```
import requests
import gradio as gr


def ask_question(query):
  try:
    response = requests.post(
      "http://127.0.0.1:8000/ask",
      json={
        "query": query
      },
      timeout=300
    )

    result = response.json()

    answer = result["answer"]

    confidence = (
      f"{result['confidence']}%"
    )

    rows = []

    for c in result["citations"]:

      rows.append([
        c["rank"],
        c["case_title"],
        c["score"],
        c["source"]
      ])
```

```
    return (
        answer,
        confidence,
        rows
    )

  except Exception as e:
    return (
        f"API Error: {e}",
        "0%",
        []
    )

with gr.Blocks(
    theme=gr.themes.Soft(),
    title="Legal AI Assistant"
) as demo:

    gr.Markdown(
        """
# ⚖️ Legal AI Assistant

### Indian Case Law Research & Reasoning System

Ask legal questions and receive answers grounded in retrieved case law.
"""
    )

    with gr.Row():

        with gr.Column(scale=3):

            query = gr.Textbox(
                label="Legal Question",
                placeholder="e.g. Can an employer terminate an employee without notice?",
                lines=4
            )

            gr.Examples(
                examples=[
                    ["Can an employer terminate an employee without notice?"],
                    ["What constitutes unfair dismissal?"],
                    ["Can a contract be terminated unilaterally?"],
                    ["What are employee rights after termination?"]
                ],
                inputs=query
            )
```

```
            submit = gr.Button(
                "Analyze Legal Question",
                variant="primary"
            )

        with gr.Column(scale=1):

            gr.Markdown(
                """
### System Information

**Corpus Statistics**

- Cases Indexed: 117
- Chunks Indexed: 4721

**Models**

- Embeddings: LegalBERT
- LLM: Llama3:8b
- Retrieval: Hybrid Search

**Backend**

- FastAPI

"""
            )


    with gr.Column():

        answer = gr.Textbox(
            label="⚖️ Legal Analysis",
            lines=20,
            scale=4
        )

        confidence = gr.Textbox(
            label="Answer Confidence",
            interactive=False,
            scale=1
        )
        gr.Markdown(
            """
            Confidence is estimated from the relevance
            scores of the retrieved legal authorities.
            """
```

```
        )


    citations = gr.Dataframe(
        headers=[
            "Rank",
            "Case Title",
            "Relevance Score",
            "Source"
        ],
        label="Supporting Authorities"
    )

    with gr.Accordion(
        "How the System Works",
        open=False
    ):

        gr.Markdown(
            """
1. User submits a legal question.

2. Hybrid retrieval searches the legal corpus.

3. LegalBERT identifies semantically relevant judgments.

4. Relevant legal context is assembled.

5. Phi-3 generates legal reasoning.

6. Supporting authorities are displayed.
"""
        )

    submit.click(
        fn=ask_question,
        inputs=query,
        outputs=[
            answer,
            confidence,
            citations
        ]
    )

    gr.Markdown(
        """
---
**Powered by:** LegalBERT • ChromaDB • Ollama (Llama3:8b) • FastAPI • Gradio
```

```
"""
    )

demo.launch(inbrowser=True)
```

api.py:

```
from fastapi import FastAPI

from src.reasoning.legal_rag_pipeline import (
    LegalRAGPipeline
)

app = FastAPI(
    title="LegalRAG API",
    version="1.0"
)

pipeline = LegalRAGPipeline()

@app.get("/")
def root():

    return {
        "message": "LegalRAG API Running"
    }

@app.post("/ask")
def ask_question(payload: dict):

    query = payload["query"]

    result = (
        pipeline.answer_query(query)
    )

    return result
```

OUTPUT SCREENSHOTS:



